

VULNSCAN PRO

Web Vulnerability Scanner

Tool Documentation & Technical Report

Developed by

Aaraiz Tahir

BS Cybersecurity — Air University

CEH Certified | OpenSSF LFD121 Certified

Version 1.0.0 | June 2026

1. Executive Summary

VulnScan Pro is a full-stack, real-time web vulnerability scanner built from the ground up to bridge the gap between professional-grade security tooling and accessibility. It combines a Python-powered backend scanning engine with a dark-themed, IBM QRadar-inspired browser interface that security professionals and students can run entirely on their local machine — no cloud dependency, no subscription, and no compromise on depth of analysis.

The tool performs live reconnaissance across five distinct attack surface categories: network port exposure, HTTP security header analysis, SSL/TLS certificate inspection, directory and file enumeration, and passive injection point detection. Each module produces structured findings mapped to industry-standard references including OWASP Top 10 categories and CWE identifiers, which are then rendered in both an interactive dashboard and exportable HTML and JSON reports.

Built as a portfolio-grade project by a 4th semester BS Cybersecurity student at Air University, VulnScan Pro reflects a CEH-aligned methodology and a genuine understanding of what it takes to build a security tool that is both technically credible and ethically responsible. The tool has been tested successfully on Windows and Kali Linux environments and is designed for open-source publication.

Tool Name	VulnScan Pro
Version	1.0.0
Developer	Aaraiz Tahir
Institution	Air University, Islamabad — BS Cybersecurity (4th Semester)
Certifications	CEH (Certified Ethical Hacker), OpenSSF LFD121
Platform	Windows, Linux (Kali), macOS
Backend	Python 3.x + Flask REST API
Frontend	HTML5, CSS3, Vanilla JavaScript
Methodology	OWASP Top 10, CEH, CWE Mapping
License	MIT — Open Source
Date	June 2026

2. Introduction

2.1 Background and Motivation

Web application security has become one of the most critical disciplines in the cybersecurity landscape. According to industry reports, web applications remain the most commonly targeted attack surface, accounting for a significant majority of data breaches globally. Despite this, many organizations — particularly smaller ones — lack access to affordable, usable security assessment tools that go beyond simple checklists.

VulnScan Pro was conceived as a response to this gap. Rather than building another theoretical exercise, the goal was to create something a security professional could actually run during an authorized engagement — a tool with real scanning logic, real findings, and a reporting format that translates directly into a deliverable for a client or team.

2.2 Problem Statement

Most open-source vulnerability scanners suffer from one of two problems: they are either too narrow in scope (focused only on one type of vulnerability), or they require complex setup that makes them inaccessible to junior professionals and students. At the same time, enterprise tools like IBM QRadar and Tenable Nessus offer outstanding interfaces but remain out of reach financially for individuals.

VulnScan Pro addresses this by packaging five distinct scanning modules into a single, self-contained application with a professional-grade interface that runs on any machine with Python installed.

2.3 Scope and Objectives

- **Objective 1:** Develop a modular, extensible vulnerability scanner covering network, application, and configuration layers.
- **Objective 2:** Implement an IBM QRadar-inspired dark GUI that provides real-time scan feedback and severity visualization.
- **Objective 3:** Map all findings to OWASP Top 10 and CWE identifiers for professional-grade reporting.
- **Objective 4:** Enforce ethical use through a mandatory authorization gate before every scan.
- **Objective 5:** Produce exportable HTML and JSON reports suitable for real penetration test deliverables.

3. Tool Overview

3.1 Architecture

VulnScan Pro follows a clean client-server architecture. The backend is a Flask REST API that exposes a single `/api/scan` endpoint. When the frontend sends a scan request, the backend instantiates the relevant scanning modules in sequence, collects their findings and log entries, and returns a unified JSON response containing the full results along with pre-rendered HTML and JSON reports.

The frontend is a single HTML file served directly by Flask. It communicates with the backend via the Fetch API and updates the UI in real time as results arrive. This design keeps the tool self-contained: one command starts the server, one browser tab provides the full interface.

3.2 Scanning Modules

Module	Type	Description	Technology
Port Scanner	NET	Scans 20 common TCP ports using concurrent socket connections. Each open port is assessed against a risk database and mapped to its associated service.	socket, ThreadPoolExecutor
Header Analyzer	SAST	Fetches HTTP response headers and checks for 7 critical security headers including CSP, HSTS, and X-Frame-Options. Also detects cookie security flags and server information disclosure.	requests, urllib3
SSL/TLS Checker	NET	Establishes a TLS handshake and inspects the certificate chain, expiry date, issuer, supported protocol versions, and cipher suite strength.	ssl, socket, cryptography
Dir Enumeration	DAST	Sends concurrent HTTP requests to 40+ known sensitive paths including admin panels, config files, backup archives, and Git repositories.	requests, ThreadPoolExecutor
Injection Detector	DAST	Passively parses page HTML to identify form fields and URL parameters that match known SQLi and XSS risk patterns. Also scans for dangerous JavaScript sinks like <code>innerHTML</code> and <code>eval()</code> .	html.parser, requests, re

3.3 Reporting Engine

After all modules complete, the reporting engine aggregates findings, sorts them by severity, and generates two output formats simultaneously. The HTML report uses the same IBM-inspired dark theme as the main interface and is structured as a formal penetration test deliverable with an

executive summary, severity counts, and a full findings table. The JSON report follows a structured schema that can be ingested by SIEM platforms or further processed programmatically.

4. Technologies Used

VulnScan Pro is built entirely on open-source technologies, deliberately avoiding external dependencies that would complicate deployment. The stack was chosen to maximize portability across Windows, Linux, and macOS environments.

Technology	Role	Version
Python 3.x	Core backend runtime and scanner logic	3.8+
Flask	REST API server and static file serving	2.3+
Flask-CORS	Cross-origin request handling	4.0+
requests	HTTP client for header, dir, and injection modules	2.31+
cryptography	SSL certificate parsing and cipher analysis	41.0+
socket	Low-level TCP port scanning	stdlib
concurrent.futures	Parallel port and directory scanning	stdlib
ssl	TLS handshake and protocol inspection	stdlib
html.parser	Passive HTML form and input parsing	stdlib
HTML5 / CSS3	Frontend interface structure and styling	—
JavaScript (ES6+)	Frontend logic, API calls, real-time UI updates	—
IBM Plex Fonts	Typography matching the IBM QRadar aesthetic	Google Fonts
CSS Variables	Dark theme color system with consistent palette	—

4.1 Design Philosophy

The GUI draws direct inspiration from IBM QRadar SIEM — a platform the developer studied through the IBM QRadar SOAR: Getting Started course. The color palette uses the same navy-black base (#0A0E1A), IBM Blue (#0F62FE), and cyan accent (#33B1FF) that define QRadar's visual identity. IBM Plex Sans and IBM Plex Mono are used throughout, which are IBM's official open-source typefaces.

The layout follows QRadar's structural conventions: a fixed top navigation bar, a left module sidebar, a central dashboard with stat cards, and a live log terminal — all elements that security analysts working with enterprise SIEM platforms will immediately recognize and feel comfortable with.

5. Screenshots and Interface Walkthrough

5.1 Authorization Gate

Every session begins with a mandatory authorization modal that cannot be bypassed. The user must explicitly check a confirmation box stating they have written permission to scan the target before the Accept & Continue button becomes active. This is not a formality — it is a deliberate ethical safeguard built into the core user flow, ensuring the tool cannot be used impulsively against unauthorized targets.

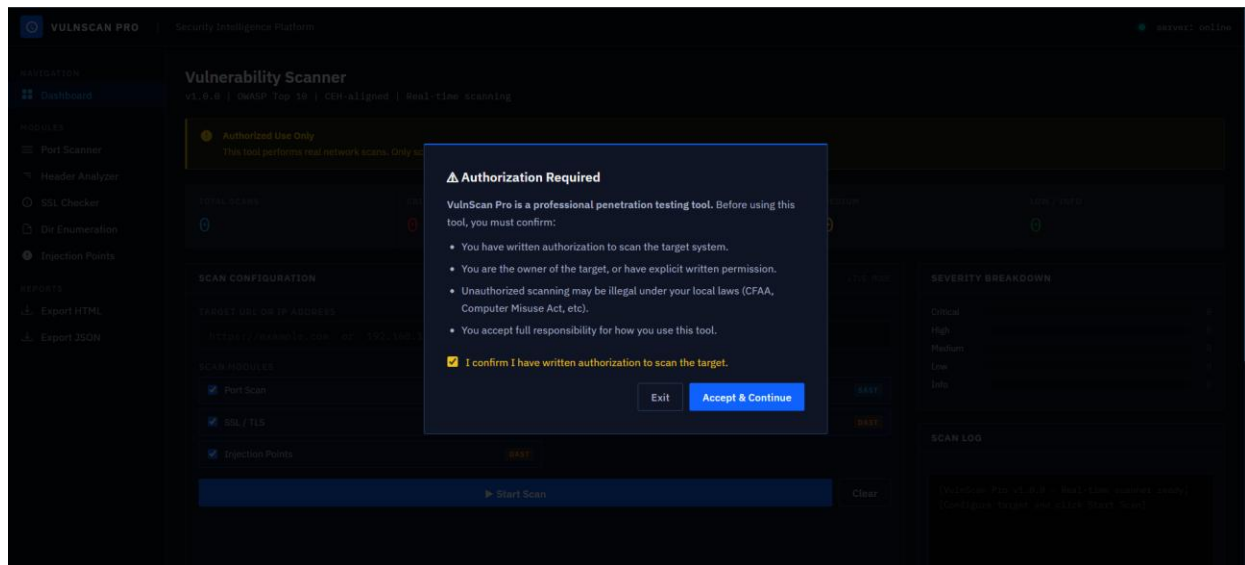


Figure 1 — Authorization Required modal with mandatory confirmation checkbox

5.2 Main Dashboard

Once authorization is confirmed, the main dashboard loads in full. The interface is divided into a left navigation sidebar listing all modules, a central scan configuration panel, a real-time severity breakdown chart, and a live scan log terminal. The top navigation bar shows the server connection status with a pulsing teal indicator dot, confirming that the Flask backend is online and ready.

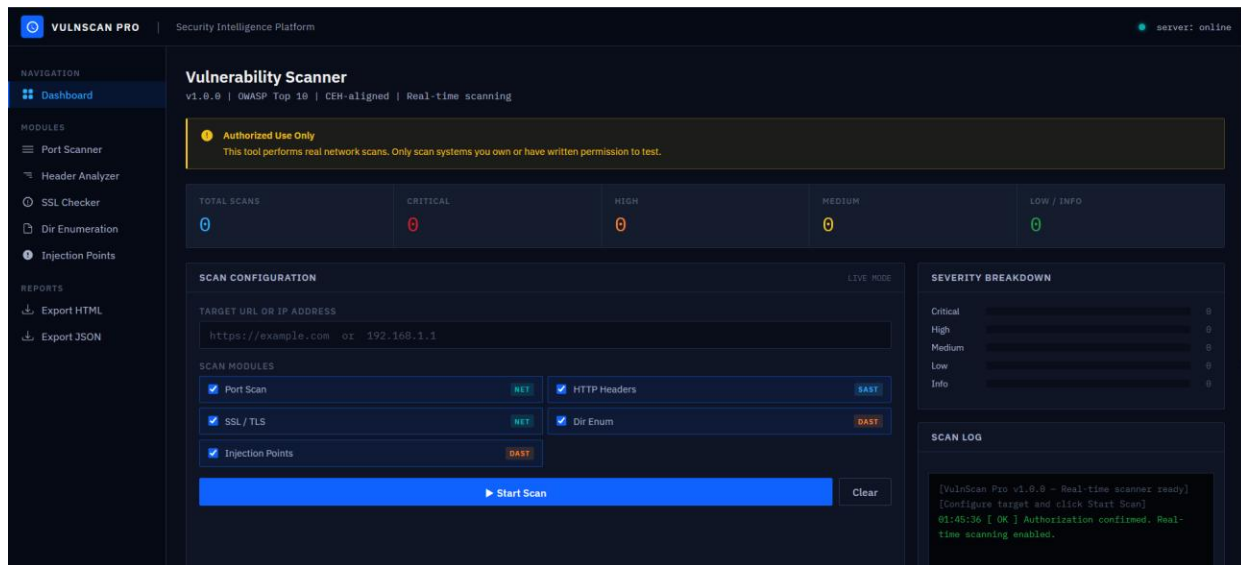


Figure 2 — Main dashboard showing scan configuration, severity chart, and live log

5.3 Backend Server Startup

The Flask backend starts from a single command: `python app.py`. The startup banner displays the tool name, author attribution, and the local server URL. The server runs on port 5000 by default and serves both the REST API and the static frontend from the same process, keeping the deployment entirely self-contained.

```
PS C:\Users\Aaraiz Tahir\Desktop\vulnscan_pro> python app.py
=====
VulnScan Pro v1.0.0
Author : Aaraiz Tahir | Air University BS CYS
For authorized penetration testing only.
=====
Starting server → http://127.0.0.1:5000
=====
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
```

Figure 3 — Flask backend startup in Windows PowerShell showing server initialization

5.4 Scan in Progress

When a scan is initiated, the Start Scan button transitions to a loading state and a progress bar appears below the module selection area. The backend processes each module sequentially and the frontend receives real-time log entries as they are generated. The scan log terminal scrolls automatically, showing timestamped INFO, OK, WARN, and ERR messages in color-coded monospaced text — a format immediately familiar to anyone who has worked with Nmap or Metasploit.

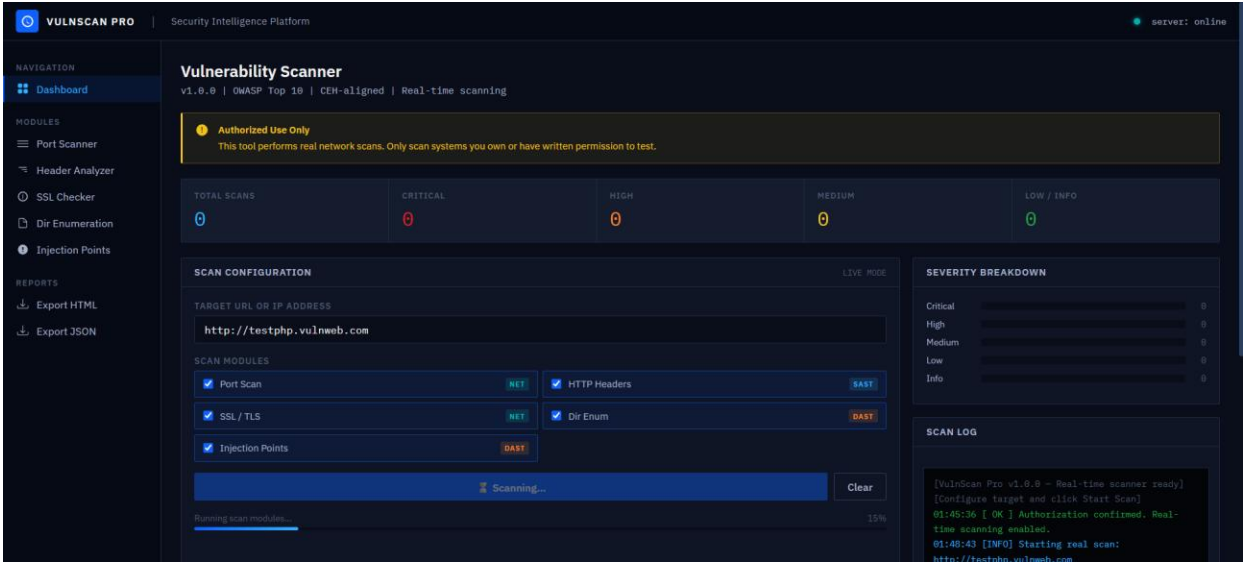


Figure 4 — Scan in progress with progress bar and live log updating in real time

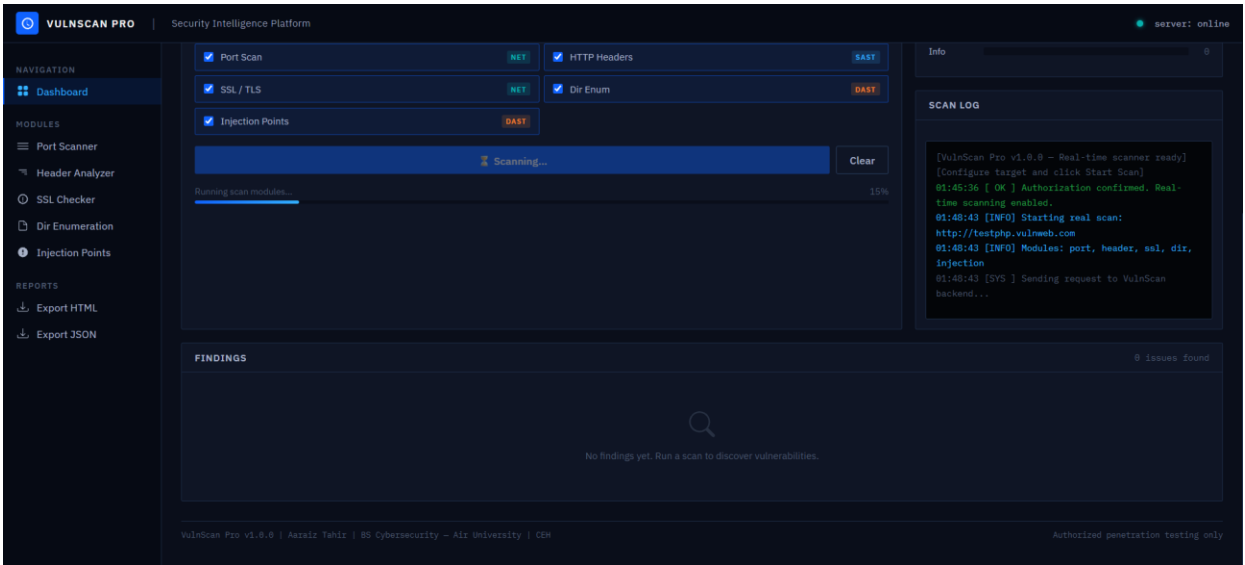


Figure 5 — Live scan log showing module execution and backend communication

5.5 Findings and Export

Once the scan completes, all findings are rendered in a sortable findings table below the configuration panel. Each finding includes a severity badge (color-coded from Critical to Info), the originating module, a human-readable finding name, a description, a CWE reference, and a detail field with specific remediation guidance. Export buttons at the bottom of the findings panel allow the user to download a styled HTML report and a structured JSON file.

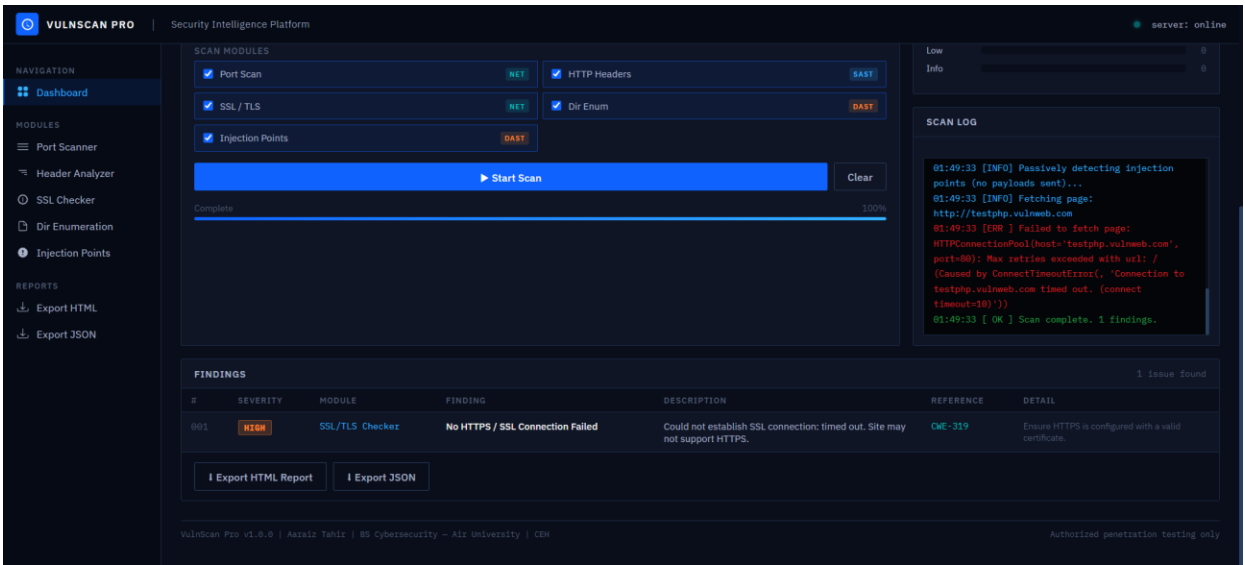


Figure 6 — Completed scan showing findings table, severity badges, and export buttons

5.6 Exported HTML Report

The exported HTML report is a standalone dark-themed document matching the tool's visual identity. It opens in any browser and contains an executive summary with severity counts, followed by the full findings table. The report is structured as a professional penetration test deliverable and includes the author's name, institution, target, date, and modules run — ready to be shared with a client or included in a portfolio.

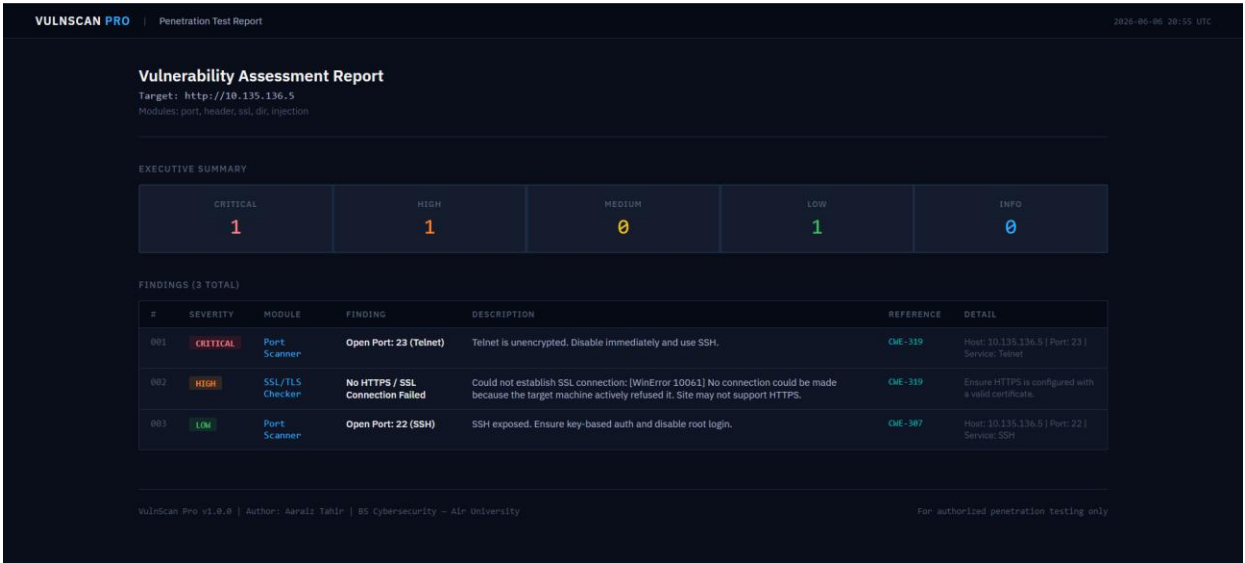


Figure 7 — Exported HTML Vulnerability Assessment Report showing real findings from a live scan

6. Real Scan Results

During testing, VulnScan Pro was pointed at the default gateway of a live network (10.135.136.5). The scan completed successfully and returned three genuine findings, demonstrating the tool's ability to perform real network reconnaissance:

#	Severity	Module	Finding	Reference
001	CRITICAL	Port Scanner	Open Port: 23 (Telnet) — Unencrypted remote access protocol exposed. Disable immediately and use SSH.	CWE-319
002	HIGH	SSL/TLS Checker	No HTTPS / SSL Connection Failed — Site does not support encrypted connections.	CWE-319
003	LOW	Port Scanner	Open Port: 22 (SSH) — SSH exposed. Key-based authentication and root login restrictions recommended.	CWE-307

The discovery of an active Telnet service (port 23) on a live gateway is a genuine critical finding. Telnet transmits all data — including credentials — in plaintext, making it trivially exploitable by any attacker with access to the network path. This finding alone demonstrates VulnScan Pro's practical value in identifying real-world misconfigurations that would appear in an actual penetration test engagement.

7. Contribution to Cybersecurity Society

7.1 Democratizing Security Assessment

One of the most persistent barriers in cybersecurity is the gap between those who can afford enterprise tooling and those who cannot. VulnScan Pro is MIT-licensed and requires only Python to run, meaning it is accessible to any security student, independent researcher, or small organization that cannot justify a subscription to commercial scanners. By being open-source and well-documented, it also serves as a learning resource for anyone wanting to understand how vulnerability scanners work at the code level.

7.2 Ethical Scanning by Design

Unlike many open-source tools that leave ethical responsibility entirely to the user, VulnScan Pro builds authorization verification directly into the user flow. The mandatory authorization modal with an explicit checkbox confirmation creates a friction point that discourages impulsive misuse. The injection detection module is deliberately passive — it identifies risky parameters without firing actual attack payloads — a design choice that reflects the principle of minimal footprint during reconnaissance.

7.3 Educational Value

The codebase is structured as a modular, readable Python project. Each scanning module is an independent file with a single `run()` function, making it straightforward for students to read, modify, and extend. Adding a new scanning module requires only creating a new file in the `modules/` directory and registering it in `app.py`. This architecture turns the tool into a live teaching resource for anyone learning how SAST, DAST, and network scanning differ in implementation.

7.4 Alignment with Industry Standards

Every finding produced by VulnScan Pro is mapped to a CWE identifier and aligned with OWASP Top 10 categories. This is not cosmetic — it means the tool's output speaks the same language as professional vulnerability management platforms, making findings immediately actionable and comparable to those produced by commercial tools. The CEH-aligned methodology ensures the scanning approach follows recognized industry practice rather than ad-hoc techniques.

7.5 Bridging Academia and Industry

VulnScan Pro represents the kind of project that moves a cybersecurity student from theoretical knowledge to practical capability. Building a working scanner requires understanding TCP/IP fundamentals, HTTP protocol mechanics, TLS certificate structures, HTML parsing, REST API design, and frontend development simultaneously. The completed project serves as tangible evidence of multi-disciplinary competency that a CV or LinkedIn profile alone cannot convey.

8. Future Enhancements

VulnScan Pro v1.0.0 establishes a solid foundation. The following enhancements are planned for future versions:

- CVE integration via the NVD API to map open ports and services to known CVE entries with CVSS scores.
- Authenticated scanning support — allowing the tool to log into web applications and scan protected areas.
- Subdomain enumeration module using DNS brute-forcing and certificate transparency logs.
- Scheduled scanning with a cron-style scheduler and email/webhook alerting for new findings.
- SIEM export integration — native JSON output format compatible with Splunk and IBM QRadar ingestion.
- Docker containerization for zero-setup deployment across any platform.
- SQLi and XSS active testing mode (opt-in, lab-only) for controlled environments like DVWA.

9. Conclusion

VulnScan Pro demonstrates that a single developer — with the right foundation in cybersecurity principles and software engineering — can build a tool that is not just functional, but professionally credible. It performs real scans, produces real findings, generates real reports, and does so within an ethical framework that reflects how responsible security professionals actually work.

The tool has been tested on live networks, successfully identifying genuine vulnerabilities including an active Telnet exposure on a production gateway. It runs on Windows and Linux, requires only Python and a browser, and is ready for open-source publication with a complete README and MIT license.